Name: R. S. Kavinadhithya
Reg. No: 192511365
course: Computer Architecture
code: CSA1210

1. Addition and subtraction using 2's complement.

Given:

Number 1 = +45

Number 2 = -23

Representation = 8-bit 2's complement

Soln

45 = 0010111

23 = 00010111

-23 = 11101000

2's = 11101000
      +1

-23 = 11101001

Addition:

  00101101
  11101001

100010110

Ignore the carry out.

00010110 = 22

Result:

45 + (-23) = 22

Subtraction:

45 - (-23)

= 45 + 23

$$\begin{array}{r} 00101101 \\ 00010111 \\ \hline 01000100 \end{array}$$

01000100 = 64 + 4

= 68

Result:

45 - (-23) = 68.

overflow = No:

2. 4- bit carry look Ahead Adder (CLA) - step by step:

A = 1010

B = 0110

Initial carry (c_0) = 0

Bit	A	B
$A_3 B_3$	1	0
$A_2 B_2$	0	1
$A_1 B_1$	1	1
$A_0 B_0$	0	0

Formula :

Generate (G) = A x B (AND)

Propagate (P) = A ⊕ B (XOR)

Bit	A	B	$G = A \cdot B$	$P = A \oplus B$
0	0	0	0	0
1	1	1	1	0
2	0	1	0	1
3	1	0	0	1

Therefore,

G = 0010

P = 1100

$C_1 = G_0 + P_0 C_0$
$= 0 + (0 \times 0)$
$= 0$

$C_2 = G_1 + P_1 C_1$
$= 1 + (0 \times 0)$
$= 1$

$C_3 = G_2 + P_2 C_2$
$= 0 + (1 \times 1)$
$= 1$

$C_4 = G_3 + P_3 C_3$

$= 0 + (1 \times 1)$

$= 1$

So,

$C_1 = 0, \quad C_2 = 1, \quad C_3 = 1, \quad C_4 = 1$

$S = P \oplus Carry$

S_0

$S_0 = P_0 \oplus C_0$

$= 0 \oplus 0$

$= 0$

$S_1 = P_1 \oplus C_1$

$= 0 \oplus 0$

$= 0$

$S_2 = P_2 \oplus C_2$

$= 1 \oplus 1$

$= 0$

$S_3 = P_3 \oplus C_3$

$= 1 \oplus 1$

$= 0$

Sum = 0000

Carry out $(C_4) = 1$

Ripple carry Adder	Carry look-Ahead Adder
Carry propagates one bit at a time	carries are calculated simultaneously
More delay	less delay
Slower	Faster
Simple hardware	More complex hardware.

Why CLA is Faster?

* It computes Generate (G) and propagate [P] Signals first.

* Carries are calculated in parallel, instead of waiting for the previous carry.

* This greatly reduces propagation delay, making CLA suitable for high-speed processors.

Generate (G) = 0010

Propagate (P) = 1100

carry = $C_1 = 0$, $C_2 = 1$, $C_3 = 1$, $C_4 = 1$

Sum = 0000

Final Result = 10000_2 (16_{10}).

3. Booth's Algorithm (-6 x +5).

Multiplicand (M) = -6

Multiplier (Q) = +5

Use 4 bit 2's complement

Multiplicand (-6)

+6 = 0110

1's = 1001

2's = 1001
      + 1

      1010

M = 1010

Multiplier (+5)

Q = 0101

Initialize Registers:

A = 0000

Q = 0101

M = 1010

Q-1 = 0

Number of bits = 4, so perform 4 cycles.

Booth's Rules

Q_0 Q_{-1}	Operation
00	100 operation
01	A = A + M
10	A = A - M
11	100 operation

Current:

A = 0000

Q = 0101

Q-1 = 0

$Q_0Q_{-1} = 10$

$A = A - M$

Since $M = 1010$ (-6)

$-M = +6 = 0110$

$$\begin{array}{r} 0000 \\ 0110 \\ \hline 0110 \end{array}$$

$A = 0110$

$Q = 0101$

$Q_{-1} = 0$

Arithmetic Right shift

$A = 0011$

$Q = 0010$

$Q_{-1} = 1$

$Q_0Q_{-1} = 01$

$A = A + M$

$$\begin{array}{r} 0011 \\ 1010 \\ \hline 1101 \end{array}$$

$A = 1110$

$Q = 1001$

$Q_{-1} = 0$

$Q_0Q_{-1} = 10$

$A = A - M$

```
  1110
  0110
------
  0100
```

$A = 0010$

$Q = 0100$

$Q_{-1} = 1$

$Q_0Q_{-1} = 01$

$A = A + M$

```
  0010
  1010
------
  1100
```

$A = 1110$

$Q = 0010$

$Q_{-1} = 0$

Combine A and Q

$AQ = 11100010$

This is an 8-bit 2's complement number.

convert to decimal:

$11100010_2 = -30_{10}$

$-6 \times 5 = 30$

Why Booth 's Algorithm is Efficient:

* Supports Signed Multiplication directly
* Reduces the number of addition and subtraction operations.
* Uses arithmetic right shifts instead of repeated additions.
* Faster than traditional binary Multiplication, especially when the multiplier contains consecutive 1s.

Multiplicand (M) = 1010

Multiplier (Q) = 0101

Final Product (AQ) = 11100010

Decimal Result = -30

4. Restoring and Non-Restoring Division:

Dividend = 13 = 1101_2

Divisor = 3 = 0011_2

Number of bits = 4

Initialize Registers :

Dividend (Q) = 1101

Divisor (M) = 0011

Accumulator (A) = 0000

Shift left

Shift (A, Q) left by 1 bit:

A = 0001

Q = 1010

Subtract Divisor:

$$\begin{array}{r} 0001 \\ -\ 0011 \\ \hline 1110 \end{array}$$ (negative)

Since A is Negative:

Restore A by adding divisor

$$\begin{array}{r} 1110 \\ 0101 \\ \hline 0001 \end{array}$$

set $Q_0 = 0$

Shift left Again:

A = 0011

Q = 0100

Subtract divisor:

$$\begin{array}{r} 0011 \\ -\ 0011 \\ \hline 0000 \end{array}$$ (negative)

Restore :

$$\begin{array}{r} 1101 \\ +\ 0011 \\ \hline 0000 \end{array}$$

Set $Q_0 = 0$

Shift left

$A = 0001$

$Q = 0010$

Subtract divisor :

$$\begin{array}{r} 0001 \\ -\ 0011 \\ \hline 1110 \end{array}$$ (Negative)

Restore :

$$\begin{array}{r} 1110 \\ 0011 \\ \hline 0001 \end{array}$$

Set $Q_0 = 0$

Quotient (Q) = 0100 = 4

Remainder (A) = 0001 = 1

Ans : $13 \div 3 = 4$, Remainder = 1

Non-Restoring Division:

Initialize:

A = 0000

Q = 1101

M = 0011

* Shift left
* Since A is positive, subtract M.
* If result is Negative, do not restore immediately.
* Instead, in ~~the~~ Next cycle, add M
* Repeat Until all 4 cycles are completed.
* After the final correction:

Quotient = 0100 0 | 4

Remainder = 0001 | 1

Why Non-Restoring in Preferred:

* Requires fewer addition/subtraction operations.
* No restoring step after every negative remainder.
* Faster execution

* widely used in modern processors for better performance:

Restoring Division	Non-Restoring Division
Restores remainder after every negative result	Does not restore immediately
More arithmetic operations	Fewer arithmetic operations.
Slower	Faster
Simpler algorithm	More efficient algorithm

5. IEEE 754 Floating Point Representation:

Represent the decimal number - 13.25 in:

* IEEE 754 Single Precision (32-bit)
* IEEE 754 Double Precision (64-bit)

Also explain floating point addition

Integer Part:

13 ÷ 2 = 6 remainder 1

6 ÷ 2 = 3 remainder 0

3 ÷ 2 = 1 remainder 1

1 ÷ 2 = 0 remainder 1

$13 = 1101_2$

Fractional part

Multiply by 2:

$0.25 \times 2 = 0.50 \rightarrow 0$

$0.50 \times 2 = 1.00 \rightarrow 1$

Therefore,

$0.25 = 0.01_2$

Binary Representation

$13.25 = 1101.01_2$

Since the number is negative.

$-13.25 = -1101.01_2$

Normalize the Number

Move the binary point after the first 1

1101.01_2

$= 1.10101 \times 2^3$

therefore,

Mantissa = 10101

Exponent = 3

IEEE 754 Single Precision (32 bit)

Sign bit:

Negative number:

Sign = 1

Exponent : Bias = 127

= 3 + 127 = 130

130 in binary

130 = 10000010

Mantissa : Remove the leading 1.

10101000000000000000000

23 bits

Final 32-bit Representation

Sign	Exponent	Mantissa
1	10000010	10101000000000000000000

Floating-Point Addition:

* Add two floating point number
* compare the exponents
* Shift the mantissa of the smaller exponent until both exponents are equal
* Add (or subtract) the mantissas
* Normalize the result.

* Round the Mantissa if necessary
* Store the final Value as:

  Sign bit
  Exponent
  Mantissa.